

Backend Requirements Addendum

Volunteer Organization Platform — Delta vs. Backend-Requirements-Document.pdf

- Reference document: docs/Backend-Requirements-Document.pdf (dated 2026-08-09)
- Source of truth: fresh static analysis of the current React/Vite frontend codebase (c:\Users\hp\Desktop\volunteer-organization), same methodology as the reference document — services, hooks, validation schemas, constants, components. File citations (path:line) point at the current code.
- Scope: this file contains only what is new, changed, or contradictory since the reference document. Anything in the reference document that still matches the current code is intentionally omitted — see the reference document itself for everything unchanged.
- Section numbers below (§) match the reference document's numbering so the two can be merged directly.

Revision history: initial pass (2026-08-09–2026-08-14); two same-day follow-up sessions on 2026-08-14 (notification read-state handling, then the applicant email field, §17.4); a verbal-confirmation follow-up on photo/logo upload scope (§17.6, appended as a continuation page); and a fourth pass on 2026-08-16 covering five further gaps found by systematic re-scan (§3.8 update, new §5.5 note, new §10 note, new §12, §17.4 update). Entries below are tagged with their session date where it isn't the initial pass.

§3.8 Governorate/City — data populated, naming ambiguity resolved, one seed-data conflict still open

NEW nameAr is now populated with real Arabic names for all 14 seed governorates (previously the field existed in the data model per the reference doc but held no data anywhere in the static list). Usable directly as backend seed data:

src/services/syrianGovernorates.js:16-31

CHANGED / RESOLVED the reference document's §23 flagged unresolved "naming drift" between "Rural Damascus" and "Rif Dimashq" across 4 duplicated governorate lists. This is now resolved app-wide, consistently, everywhere a governorate/city value is actually *submitted*: the canonical governorate is displayed as "Rural Damascus" / "Rif Dimashq (Damascus Countryside)", but the value transmitted to the API for that governorate is uniformly the string "Rif Dimashq" — never "Rural Damascus" — in every form:

Location	File:line
Volunteer profile validation (enum)	src/utils/auth/VolunteerProfileValidation.js:29
Volunteer profile form (select value)	src/components/volunteerProfile/ProfileForm.jsx:27,40
Organization profile validation (enum)	src/utils/auth/OrganizationProfileValidation.js:18,24-25
Organization profile form / Opportunity creation form (select value)	src/components/OrgProfile/ProfileForm.jsx:20-22 , src/components/organization/CauseForm.jsx:12-14

Backend action: the city / location enum accepted on volunteers, organizations, and opportunities must accept "Rif Dimashq" as the value for this governorate — not "Rural Damascus". This applies identically to all three entities now (previously an open inconsistency; now a single deliberate, unified choice on the frontend).

UPDATE (2026-08-16) — Needs Decision seed-data nameEn still conflicts with the transmitted value for the same governorate

The resolution above covers every place a city value is *submitted*. It does not cover the seed-data array recommended above as backend seed data: that array's nameEn for this same governorate is still "Rural Damascus", not "Rif Dimashq" :

```
src/services/syrianGovernorates.js:18
{ id: 2, nameEn: "Rural Damascus", nameAr: "ريف دمشق", slug: "rural-damascus" }
```

Impact: if the backend team seeds the governorates table's name_en column verbatim from this array (as recommended above for name_ar) and also treats that same column as the source of truth / validation list for the city field accepted on volunteers/organizations/opportunities, a submitted city: "Rif Dimashq" would never match the seeded record name_en: "Rural Damascus" for the same place — the exact mismatch this section already resolved once, reintroduced one layer down.

Needs decision: either (a) treat the `governorates` catalog table's `name_en` as a display label independent from the separately-validated `city` string field (non-issue, but should be stated explicitly), or (b) rename this array's `nameEn` for id 2 to "Rif Dimashq" before using it as seed data, keeping `nameAr` / `slug` as-is.

§5.5 Admin self-service — frontend-internal note (no backend action)

UPDATE (2026-08-16) — frontend-internal, not a backend gap the admin "Reset password" action on the Settings page (`src/components/admin/AdminResetPasswordModal.jsx:28-38`) currently simulates sending a reset email via a hardcoded `setTimeout` and never calls a service function. This addendum's §5.4-equivalent endpoint already covers this use case: `requestPasswordReset({ email })` (`src/services/auth.js:335`, calling `POST /forgot-password`) is already implemented and already used by the public "Forgot password" page. No new endpoint is implied — listed here only so a reader merging documents doesn't mistake the modal's current fake behavior for a missing backend capability. Frontend follow-up: wire the modal's submit handler to the existing `requestPasswordReset` service.

§6 Route Map — two new routes, one legacy redirect

Path	Access	Page
<code>/my-journey</code>	Volunteer only	NEW replaces the standalone achievements page: hours summary, achievements, and a completed-opportunities timeline, all in one page (<code>src/pages/volunteerJourney.jsx</code>). No new API calls — reuses existing participations/achievements queries.
<code>/notifications</code>	Any authenticated user	NEW full notification history/filter page (<code>src/pages/notifications.jsx</code>), reached via "See all notifications" from the navbar bell.

Legacy redirect added: `/achievements` → `/my-journey` (`src/constants/paths.js:13,21,39`).

DOC/CODE MISMATCH the reference document's §6 lists the applicants route as `/applicants/:id`. The actual registered route (unchanged, already true when the reference document was written) is `/mycauses/applicants/:id` (`src/constants/paths.js:17`, `src/App.jsx:95`). Worth correcting in the merged route map.

§10 Volunteer Requirements — education-level mismatch resolved, in the opposite direction the reference doc recommended

UPDATE (2026-08-16) — supersedes reference doc recommendation the reference document's §10 (and its §20 validation table, and §23-B frontend-internal inconsistencies) flagged that the Zod schema allowed 6 education levels while the profile form only rendered 4 (Master's/PhD commented out), and recommended "backend should accept all 6 regardless." This has since been resolved in the opposite direction: as of commit `e543e98` (2026-08-15), the Zod schema itself was cut down to match the form, via an explicit product-decision comment:

```
src/utils/auth/VolunteerProfileValidation.js:18-27
// قرار منتج مقصود: ما فينا نستبعد "ماجستير"/"دكتوراه" مش منطقي حلة
// هالشهادتين يسجلوا كمتطوعين بمنصة تطوع أصلاً (سياق المنصة مختلف عن
// سياق توظيف أكاديمي). الفورم (ProfileForm.jsx) أصلاً بيخفيهم، وهون
// بنمنعهم على مستوى الـ schema كمان حتى ما يوصلوا كقيمة صالحة بأي طريق
const EDUCATION_LEVEL_OPTIONS = [
  'No Formal Education',
  'High School',
  'Diploma',
  "Bachelor's Degree",
]
```

(Translation: "Deliberate product decision — we can't include Master's/PhD, it doesn't make sense for holders of those degrees to register as volunteers on a volunteering platform [different context from academic employment]. The form already hides them, and here we block them at the schema level too, so they can't reach a valid value through any path.")

Both the UI (`src/components/volunteerProfile/ProfileForm.jsx:16-21`) and the validation schema now agree on exactly 4 values, resolving the mismatch — but with the opposite backend implication than originally recommended.

Backend action: the `education_level` enum only needs to accept 4 values — `No Formal Education` , `High School` , `Diploma` , `Bachelor's Degree` . `Master's Degree` and `PhD` should NOT be accepted, per this explicit product decision. This supersedes the reference document's §10/§20/§23-B recommendation to accept all 6.

§12 Admin Requirements — notification preferences have no backend endpoint

NEW (2026-08-16) the admin Settings page has a "Notification preferences" panel (three toggles: `verificationEmails` , `weeklyDigest` , `securityAlerts`) persisted exclusively to `localStorage` (`admin-settings-preferences` key) — there is no service call, no `MOCK_MODE` branch, and no real-mode request for it anywhere. This is distinct from the already-documented `PUT /admin/profile` and `PUT /admin/password` (reference document §5.5/§12) — neither of those was ever intended to carry these fields.

```
src/pages/admin/AdminSettings.jsx:17      const STORAGE_KEY = 'admin-settings-preferences'
src/pages/admin/AdminSettings.jsx:19-33  loadPreferences() — reads localStorage only
src/pages/admin/AdminSettings.jsx:41-46  // التودو: هذه التفضيلات محفوظة محليًا فقط (localStorage) حاليًا
```

Impact: these preferences do not sync across devices/browsers for the same admin account, and there is currently no server-side record of the admin's choice at all — if the backend is ever meant to gate sending verification-decision emails / weekly digests / security alerts on these toggles, today's client-side setting has no functional effect on real email sending.

Backend action (needs decision): confirm whether admin notification preferences should be a real, persisted resource (e.g. `GET/PUT /admin/preferences` , or fields on the admin/user record) before backend email-sending logic is built to respect them, or explicitly decide this stays a client-only cosmetic setting with no backend effect.

§14 Notifications — catalogue grew, one endpoint now concrete

The reference document described this system as "approved by backend but not built" with only a hypothetical shape. The frontend notification system has since been substantially built out.

NEW notification type — `applicant-new` : "New volunteer application" — fires for an organization when one of its opportunities has an unseen pending participation. Links to `/mycauses/applicants/{opportunityId}` , uses the same seen-tracker as `applicant-withdrawn` (`organizationApplicantSeenTracker.js`). `src/services/notifications.js:184-192` , `src/constants/notificationTypes.js:17,29,43` . This brings the full notification type catalogue to 9 types (reference document's §14 listed 8): `achievement` , `hours` , `status-accepted` , `status-rejected` , `org-verified` , `org-rejected` , `applicant-new` , `applicant-withdrawn` , `opportunity-reminder` .

NEW concrete mark-as-read endpoint, not just a hypothetical shape: `POST /notifications/{id}/read` — `src/services/notifications.js:278` . The reference document's §14 only speculated a PATCH-style mechanism; the frontend has committed to POST, not PATCH. `GET /notifications` itself is still unbuilt server-side.

NEW "Mark all as read" (`pages/notifications.jsx`) does not call a bulk endpoint — it loops and fires `POST /notifications/{id}/read` once per item (`src/pages/notifications.jsx:76-83`). N parallel requests, not one. A bulk `POST /notifications/read-all` would help but isn't required by the current contract.

UPDATE (2026-08-14) client-side unread filtering added, fixing a real bug: the real-mode client previously read a `seen` field back from the hypothetical `GET /notifications` response but never filtered on it. It now sends `?unread=true` optimistically and defensively filters `seen: true` items client-side regardless of backend support (`src/services/notifications.js:258-282`).

UPDATE (2026-08-14) mark-as-read now also fires on click-through, not only explicit dismiss

(`src/components/notifications/NotificationListItem.jsx:27-33`) — same endpoint/contract, higher call frequency. No new endpoint required.

Backend action: optionally support `?unread=true` on `GET /notifications` (not required — frontend filters defensively — but recommended given the client polls every 5 seconds while a session is active, `src/hooks/useRecentUpdates.js:23`).

§15 Dashboards & Analytics (Organization) — recentActivity filter changed

CHANGED the reference document's §15 defines `recentActivity` as "top 5 by date desc" with no status filter. The frontend's client-side composition now filters to pending-status applicants only before taking the top 5, to surface what still needs action rather than a general activity log. `src/services/dashboard.js:291-305`

Backend action: when `GET /organizations/me/dashboard` is implemented, `recentActivity` should be the 5 most recent *pending* participations for the organization's opportunities, ordered by date descending — not the 5 most recent participations of any status.

§17.4 Applicants list (`GET /opportunities/{id}/participants`) — two rounds of new required fields

UPDATE (2026-08-14) the applicant card and volunteer profile preview modal now display the volunteer's email next to their phone number, read directly off the nested `volunteer` object, same treatment as the existing `volunteer.phone` / `volunteer.photo`. Exact field: `volunteer.email`, sibling to the already-documented `volunteer.phone` / `volunteer.photo` (reference doc §16) — not a top-level field on the participation record.

Backend action: add `email` to the nested `volunteer` object on `GET /opportunities/{id}/participants` (sourced from `users.email`). If absent, the frontend simply omits the row (same defensive pattern as `volunteer.city` / `volunteer.phone`) — safe to ship incrementally. `src/components/organization/ApplicantCard.jsx:93-98`, `src/components/organization/VolunteerProfilePreviewModal.jsx:115-123`; unchanged: `src/services/participations.js:129-131`.

UPDATE (2026-08-16) — High, confirmed gap nested `volunteer` object also needs org-scoped stats + achievements

Direct inspection of the components that consume this same response shows three more fields on the nested `volunteer` object that neither the reference document nor the rest of this addendum mention, and that are not part of the base Volunteer profile entity (reference doc §3.2):

Field	Type	Meaning
<code>completedOpportunitiesCount</code>	number	Opportunities the volunteer completed with this specific organization only — not a platform total
<code>totalHoursVolunteered</code>	number	Confirmed hours logged with this specific organization only — not a platform total
<code>achievements</code>	Achievement[]	The volunteer's full platform-wide achievement catalog (same shape as <code>GET /volunteers/{id}/achievements</code>)

Both stat cards and the achievements list are rendered directly from the `volunteer` object passed into the applicant card/modal — there is no separate API call in real mode:

- `src/components/organization/VolunteerProfilePreviewModal.jsx:59-61` — renders `volunteer.totalHoursVolunteered ?? 0`, `volunteer.completedOpportunitiesCount ?? 0`, and achievements from `volunteer.achievements || []`, explicitly labeled in the UI as "Hours and completed opportunities are with your organization specifically. Achievements are earned across the whole platform."
- `src/components/organization/ApplicantCard.jsx:29,49` — receives the same object, passes it straight to the modal; no additional fetch.
- `src/services/participations.js:85-116` (mock branch of `fetchApplicantsForOpportunity`) shows the required scoping: it filters ALL of the volunteer's participations platform-wide down to the entry matching the *current* `organizationId` before reading `completedOpportunitiesCount` / `confirmedHours`, then separately fetches the volunteer's full platform-wide achievement list.

Impact: since the frontend makes no per-applicant follow-up calls in real mode, if the backend's response doesn't embed these three fields inside each item's `volunteer` object, every applicant card/profile-preview will silently show "0 hours here", "0 completed here", and no achievements — no error, just wrong/empty data for every organization reviewing applicants.

Backend action: the nested `volunteer` object on each participant row must additionally include `completedOpportunitiesCount` and `totalHoursVolunteered` computed **scoped to the requesting organization's own**

opportunities only (filtered by `organization_id`, not a platform-wide sum), plus the volunteer's full `achievements` array (platform-wide, same computation as the achievements endpoint in reference doc §13).

§19 Full Endpoint Catalog — additions

Method & Path	Request	Response	Status
POST <code>/notifications/{id}/read</code>	—	success/error	NEW verb/path now committed in frontend code, see §14
GET <code>/notifications</code>	optional <code>?unread=true</code>	array of notification objects (<code>id</code> , <code>type</code> , <code>title</code> , <code>description</code> , <code>href</code> , <code>seen</code>)	UPDATE (2026-08-14) query param is a frontend addition, optional server-side. Endpoint itself still unbuilt.
GET <code>/opportunities/{id}/participants</code>	—	applicant list; nested <code>volunteer</code> object must include <code>email</code> , <code>completedOpportunitiesCount</code> , <code>totalHoursVolunteered</code> , <code>achievements</code>	UPDATE (2026-08-14 / 2026-08-16) <code>volunteer.email</code> added 2026-08-14 (§17.4); org-scoped stats + achievements fields added 2026-08-16 (§17.4 update) — all four now required on this response.

§22 Backend Team Checklist — additions

- Ensure the governorate/city enum accepts "Rif Dimashq" (not "Rural Damascus") as the value for that governorate, consistent across volunteers, organizations, and opportunities (§3.8).
- When building the notifications mark-as-read mechanism, match the frontend's committed contract: POST `/notifications/{id}/read` per notification id (§14).
- When building GET `/organizations/me/dashboard`, filter `recentActivity` to pending-status participations only, not all statuses (§15).
- Optionally support `?unread=true` on GET `/notifications`; not required since the frontend filters defensively, but recommended given the 5-second polling interval (§14 update).
- Add `email` to the nested `volunteer` object on GET `/opportunities/{id}/participants` (sourced from `users.email`), alongside the already-expected `volunteer.phone` and `volunteer.photo` — required, no fallback (§17.4).
- **NEW (2026-08-16)** also add `completedOpportunitiesCount` and `totalHoursVolunteered` (org-scoped, not platform-wide) and the full `achievements` array to that same nested `volunteer` object — required, no fallback call exists (§17.4 update).
- **NEW (2026-08-16) — needs decision** confirm whether admin notification preferences (`verificationEmails`, `weeklyDigest`, `securityAlerts`) need a real backend-persisted resource, or stay client-only (§12).
- **NEW (2026-08-16) — needs decision** confirm whether the `governorates` seed table's `name_en` for "Rif Dimashq" should match the literal value forms submit, or is understood as a separate display-only catalog (§3.8 update).
- **NEW (2026-08-16) — supersedes earlier item** `education_level` enum should now only accept 4 values, not 6 — supersedes the original checklist's "accept all 6" recommendation (§10 update). No backend action beyond restricting to 4 values.

End of core addendum. All findings above trace to specific frontend source files; none were inferred beyond what the code and its inline comments state explicitly. Anything from the reference document not mentioned here is unchanged. Entries are tagged with their session date where later than the initial 2026-08-09–2026-08-14 pass.

Backend Requirements Addendum — Update (Photo/Logo Upload)

Volunteer Organization Platform — continuation of the addendum above

- Scope: this page covers only the profile-photo / organization-logo upload flow, confirmed in a direct conversation with the backend team. Reference: this addendum's own §17.4 (volunteer email) and the original Backend-Requirements-Document.pdf. Section numbers below continue the same §-numbering convention as the rest of this addendum.

§17.6 Profile photo / logo upload — confirmed backend scope

Backend team confirmed verbally that photo/logo upload will be fully supported across the project (previously declined when first proposed). Two separate, independent gaps were found by direct inspection of the current backend code. Both are required — unrelated bugs in different controllers, fixing one does not fix the other. Frontend is already built and ready for both; no further frontend work is required once each ships.

Gap	Status	Current backend behavior (confirmed by code)	Required fix
A. Volunteer photo field shape <code>VolunteerResource.php</code>	Confirmed by backend	<code>'photo' => \$this->getFirstMedia('profile_photo')</code> ?: null — response will be a URL per Spatie convention, but <code>getFirstMedia()</code> returns a full Spatie Media model object, not a URL string.	Change to <code>getFirstMediaUrl('profile_photo')</code> — same pattern already used correctly in <code>OrganizationResource.php</code> for <code>profile_image</code> . One-line fix.
B. Organization logo on update <code>OrganizationController.php</code>	Still required, not yet explicitly confirmed by backend	<code>store()</code> (registration) already handles <code>hasFile('photo')</code> correctly. <code>update()</code> only runs <code>\$organization->update(\$request->validated())</code> — no file handling at all. An organization cannot change its logo after registration today.	Add the same <code>hasFile('photo')</code> handling already used in <code>store()</code> to <code>update()</code> : clear the existing <code>profile_image</code> media collection and add the new file, matching <code>VolunteerController::update()</code> 's existing pattern for <code>profile_photo</code> .

Backend action — both required, independently of each other:

1. `VolunteerResource.php` — swap `getFirstMedia` → `getFirstMediaUrl` for the photo field (Gap A) — confirmed in progress.
2. `OrganizationController::update()` — add `hasFile('photo')` handling to the `profile_image` media collection, mirroring `VolunteerController::update()` (Gap B) — still needs explicit confirmation. Without this, organizations remain unable to change their logo after registration even after Gap A ships. `OrganizationRequest.php` will also need a photo validation rule added (currently has none — see original Backend-Requirements-Document.pdf's request-rules table).

Frontend readiness — confirmed, no action needed once shipped

Endpoint	Frontend status	Notes
POST <code>/volunteers/{id}</code> (<code>_method=PUT</code> , multipart)	Ready	Already sends photo as a file. Response parsing already defensively handles both a plain URL string and a raw Media object (for Gap A above), so it will pick up the fix automatically — no frontend change needed the moment Gap A ships.
POST <code>/organizations/{id}</code> (<code>_method=PUT</code> , multipart)	Sending, backend not yet processing	Frontend already sends photo as a file on every save. Backend currently ignores it silently (no error, no crash) until Gap B ships — logo changes just won't persist until then. Reads the response's <code>profile_image</code> field already, ready the moment it's populated.

Mock mode (local frontend testing without a live backend) already fully simulates both flows today, storing photos as base64 Data URLs so they survive page reloads — this was a separate, already-fixed frontend-only issue unrelated to the two backend gaps above.